



Sponsorship App - Complete Learning Guide

Welcome to Your Complete Learning Journey!

This guide will take you from zero understanding to become an expert in the Sponsorship App. We'll cover every technology, every concept, every component, and how they all work together. This is your complete reference for your interim evaluation at Cognizant.



Table of Contents

1. Project Overview & Business Logic
2. Technologies Used (Frontend & Backend)
3. Backend Architecture & Concepts
4. Frontend Architecture & Concepts
5. Database Design
6. Authentication & Security
7. Complete Data Flow
8. Components Explanation
9. User Roles & Permissions
10. Complete Workflows

1 Project Overview & Business Logic

What is the Sponsorship App?

Imagine a platform where **brands need help promoting their products** on social media. They create "campaigns" (like "promote our new shoes on Instagram"). On the other side, **influencers** (people with many social media followers) see these campaigns and apply to participate. The platform helps them communicate, agree on terms, complete the work, and handle payments. That's what the Sponsorship App does!

Three Main Players (User Roles)

1. **Admin:** Manages the entire platform, monitors users, handles disputes
2. **Brand:** Creates campaigns, reviews applications, makes payments
3. **Influencer:** Applies to campaigns, completes work, receives payments

Business Flow (Simple Explanation)

1. **Brand creates a campaign** - "We want someone to promote our shoes on Instagram for \$5,000"
 2. **Influencer applies** - "I have 100K followers and I'm interested!"
 3. **Brand reviews and approves** - "Yes, you're perfect for this!"
 4. **Influencer does the work** - Creates posts, videos, etc.
 5. **Influencer submits proof of work** - Links to posts, screenshots, analytics
 6. **Brand approves the work** - "Great job! Payment processing..."
 7. **Payment is sent** - \$5,000 transferred to the influencer
 8. **Both rate each other** - Leave feedback for future collaborations
-

2 Technologies Used

Why These Technologies?

Different technologies serve different purposes. The **backend handles logic and data**, while the **frontend handles user interface**. Think of it like a restaurant: the kitchen (backend) prepares food, and the dining room (frontend) serves it to customers.

Backend Technologies

- ✓ **Java 17** - Programming language (fast, reliable)
- ✓ **Spring Boot 3.2.5** - Framework for building backend
- ✓ **Spring Data JPA** - Works with databases
- ✓ **Spring Security** - Protects data & authentication
- ✓ **Hibernate** - Connects Java objects to database
- ✓ **MySQL 8** - Database to store data
- ✓ **JWT (jjwt)** - Secure token authentication
- ✓ **Maven** - Builds and manages project

Frontend Technologies

- ✓ **Angular 15+** - Framework for web interface
- ✓ **TypeScript** - Typed version of JavaScript
- ✓ **RxJS** - Handles asynchronous operations
- ✓ **Angular CLI** - Command line tool
- ✓ **SCSS** - Advanced CSS for styling
- ✓ **HTML5** - Page structure
- ✓ **Bootstrap/Angular Material** - UI components
- ✓ **npm** - Package manager

💡 **Key Point:** Java & Spring Boot are like the brain of the application. Angular is like the face. They communicate via HTTP (the language they use to talk to each other).

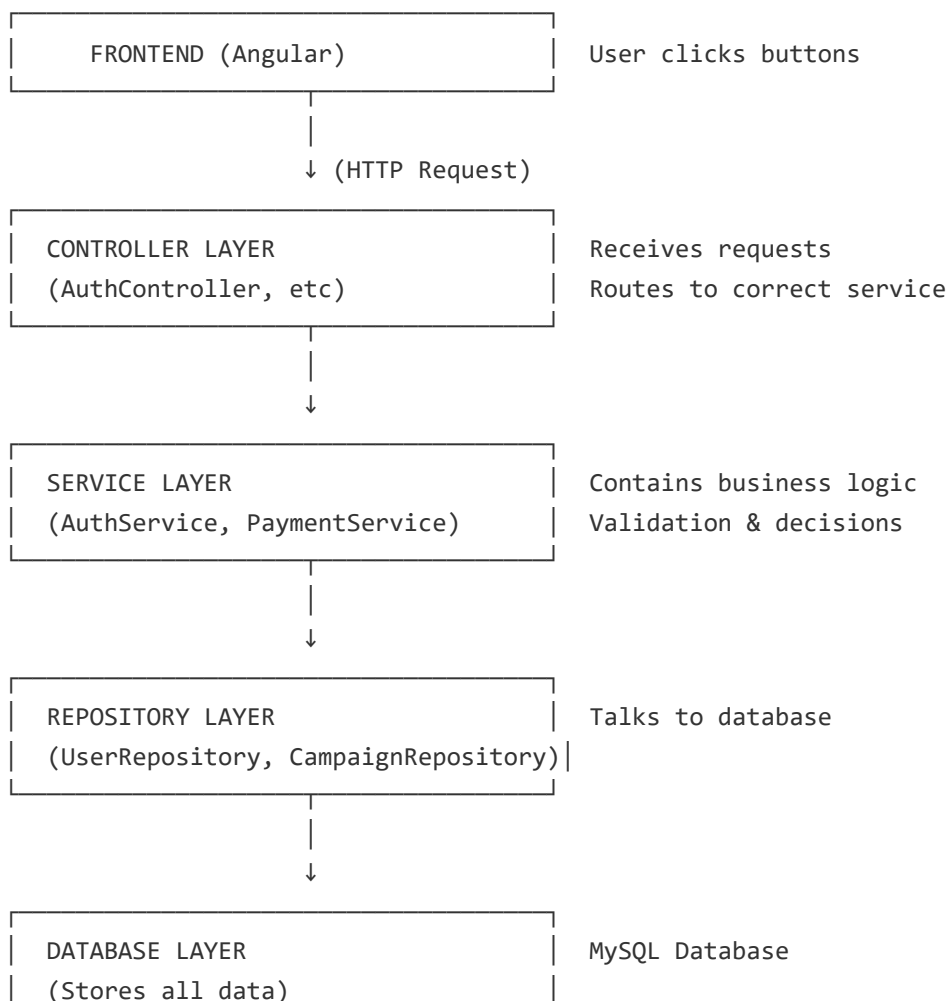
3 Backend Architecture & Concepts

What is Backend?

Backend is the "behind-the-scenes" part of your application. It's where all the thinking happens:

- Checking if user entered correct password
- Storing data in database
- Calculating payments
- Making decisions (Can this influencer apply? Can brand approve?)
- Protecting data from unauthorized access

Backend Layers (Like a Sandwich)



Layer Explanation (Simple Words)

CONTROLLER: Like a receptionist at a hotel. When you (frontend) send a request, controller receives it and says "I'll send this to the appropriate department"

SERVICE: Like the hotel manager. Contains all the rules and decisions. "Can this person check in? Do they have valid ID? Is the room available?"

REPOSITORY: Like the filing system. When you ask "Is there a user with email xyz@gmail.com?", repository searches the database and returns the answer.

DATABASE: Like a giant filing cabinet. Stores all information about users, campaigns, payments, etc.

Key Backend Components

1. Entities (Data Models)

Entities are **Java classes that represent data in the database**. Each entity is like a table in Excel:

Entity Name	What It Represents	Key Fields
User	Admin, Brand, or Influencer	id, name, email, password, role, bio
Campaign	A promotion project	id, name, description, budget, startDate, brand_id
SponsorshipRequest	Influencer application to campaign	id, influencer_id, campaign_id, status, proposal

Payment	Money transfer record	id, amount, status, campaign_id, influencer_id
Notification	Alert message to user	id, user_id, message, isRead
Rating	Review/feedback	id, campaign_id, rater_id, rated_id, score, feedback

Relationships (How Entities Connect)

One-to-Many: One User can have many Campaigns (One brand creates many campaigns)

Many-to-One: Many SponsorshipRequests belong to One Campaign

Many-to-Many (implied): Many influencers apply to many campaigns

2. Services (Business Logic)

Services contain the actual rules and decision-making. Here are the main services:

Service Name	What It Does
AuthService	Login, signup, password validation, JWT token creation
CampaignService	Create campaigns, list campaigns, update status
SponsorshipService	Handle applications, approve/reject, check work
PaymentService	Process payments, record transactions, verify funds
NotificationService	Send notifications when things happen
RatingService	Store ratings and feedback after campaign completion
AdminService	Admin-only operations, user management, reports

Backend Technology Details

Spring Boot Framework

Spring Boot is like a toolkit that provides pre-built components. Instead of building everything from scratch, you use existing tools:

- **Spring Data JPA:** Automatic database operations (no need to write SQL)
- **Spring Security:** Authentication & authorization (login, permissions)
- **Spring Web:** HTTP request handling (@RestController, @RequestMapping)
- **Validation:** Check if data is correct before saving

@Annotations (Magic Tags)

Annotations are like instructions you put on your code. They tell Spring what to do:

@SpringBootApplication - "This starts the entire application"

@Controller - "This handles web requests"

@Service - "This contains business logic"

@Repository - "This handles database"

@Entity - "This is a database table"

@Table - "Database table name is..."

@PostMapping - "Handle POST requests here"

@GetMapping - "Handle GET requests here"

Hibernate & JPA (Database Magic)

Normally, connecting to database requires writing complex SQL. Hibernate handles this automatically. You just write Java code and it creates SQL!

```
// Without Hibernate - You write SQL sql = "SELECT * FROM users  
WHERE email = 'john@gmail.com'" // With Hibernate/JPA - You write  
Java Optional<User> user =  
userRepository.findByEmail("john@gmail.com"); // Hibernate converts  
this to SQL automatically!
```



Frontend Architecture & Concepts

What is Frontend?

Frontend is what you **see and interact with** in your browser. It's the beautiful interface, the buttons you click, the forms you fill. Everything you see is frontend.

Frontend Architecture



Angular Framework Basics

Angular is a framework that helps you build interactive web applications. Think of it like LEGO blocks - you build small components and combine them.

Component (The Building Block)

A component is like a mini-application. It has:

- **.ts file (TypeScript):** Logic - what happens when you click buttons
- **.html file (HTML):** Template - what you see on screen
- **.scss file (CSS):** Styling - colors, fonts, layout

```
// Example Component Structure
import { Component } from '@angular/core';
@Component({ selector: 'app-login', // This component is called <app-login>
  templateUrl: './login.component.html', // The HTML file
  styleUrls: ['./login.component.scss'] // The CSS file })
export class LoginComponent {
  username: string = '';
  password: string = '';
  login() { // What happens when user clicks LOGIN button } }
```

Key Frontend Concepts

1. Two-Way Data Binding

When you type something in a text box, Angular automatically updates the variable. When the variable changes, the display updates. **It's like magic!**

```
// HTML Template <input [(ngModel)]="emailAddress"> // TypeScript
emailAddress: string = ''; // When user types in input, emailAddress
updates automatically! // When emailAddress changes in code, input
updates automatically!
```

2. Dependency Injection

Instead of a component creating everything it needs, Angular provides (injects) what it needs. Like ordering from a restaurant instead of cooking yourself.

```
export class LoginComponent {
  constructor(private authService: AuthService) {
    // Angular automatically provides authService! // We can now use:
    this.authService.login() } }
```

3. Services (Data & Logic Sharing)

Services are like shared libraries. Multiple components can use the same service to avoid repeating code. For example, AuthService is used by Login, Signup, and Profile components.

4. Observables & RxJS

Observables handle data that arrives over time (like network requests). Instead of waiting for response with frozen UI, Angular continues and responds when data arrives.

```
// Without Observable (UI freezes while waiting) let campaigns =
getCampaigns(); // Waits here... render(campaigns); // Only then
renders // With Observable (UI stays responsive)
getCampaigns().subscribe(campaigns => { render(campaigns); //
Renders when data arrives }); // UI continues working while waiting!
```

Frontend Components Explained

Component	Purpose	What User Does
LoginComponent	User login page	Enter email & password
SignupComponent	User registration page	Create new account
CampaignListComponent	Show all campaigns	Browse available campaigns
CampaignDetailComponent	Single campaign details	Read details, click Apply
CampaignFormComponent	Create/Edit campaign form	Fill form, submit campaign
DashboardComponent	Main page after login	See overview of activities
PaymentComponent	Payment interface	Process payment
NotificationComponent	Alert messages	See notifications

RatingComponent	Rate after campaign	Leave review & stars
------------------------	---------------------	----------------------

Frontend Services

Services in Angular handle communication with backend and share data:

Service	Responsibility
AuthService	Login, logout, token management, current user
CampaignService	Get campaigns, create campaign, update campaign
SponsorshipService	Apply to campaign, view applications, approve/reject
PaymentService	Process payments, view payment history
NotificationService	Get notifications, mark as read
RatingService	Submit ratings, view ratings

Guards (Access Control)

Guards are like security guards at a nightclub. They check "Are you logged in? Do you have the right role?" before letting you access a page.

Types of Guards

AuthGuard: Checks if user is logged in. If not, redirects to login page.

RoleGuard: Checks if user has the right role. Admin page requires ADMIN role!

5 Database Design

What is a Database?

A database is like a giant organized filing cabinet. Instead of searching through papers, you ask the database: "Show me all users with 'gmail' email" and it instantly finds them.

Database Structure

Your database has 6 main tables:

Table 1: USERS

Column	Type	Meaning
id	INT (Primary Key)	Unique identifier for each user
name	VARCHAR	User's full name
email	VARCHAR (Unique)	Email address (no duplicates)
password	VARCHAR (Encrypted)	Password (stored encrypted with BCrypt)
role	ENUM	ADMIN, BRAND, or INFLUENCER
bio	TEXT	User bio/about section
profileImage	VARCHAR	Profile picture URL

Table 2: CAMPAIGNS

Column	Type	Meaning
id	INT (Primary Key)	Unique campaign ID

name	VARCHAR	Campaign name
description	LONGTEXT	Detailed campaign info
platform	VARCHAR	Instagram, YouTube, TikTok, etc
budget	DECIMAL	Campaign budget amount
startDate	DATE	Campaign start date
endDate	DATE	Campaign end date
status	ENUM	ACTIVE, PAUSED, COMPLETED, EXPIRED
brand_id	INT (Foreign Key)	Which brand created this campaign

Table 3: SPONSORSHIP_REQUESTS

Represents when an influencer applies to a campaign

Column	Meaning
id	Unique ID
influencer_id	Which influencer applied
campaign_id	Which campaign are they applying to
proposal	Influencer's pitch (why they should be selected)
status	PENDING, ACCEPTED, REJECTED, COMPLETED
workDescription	Links to posts/videos created by influencer
workSubmittedAt	When influencer submitted their work
workCompletedAt	When brand approved the work

Table 4: PAYMENTS

Records all payment transactions

Column	Meaning
id	Payment transaction ID
campaign_id	Which campaign this payment is for
influencer_id	Who is receiving the payment
brand_id	Who is making the payment
amount	How much money
status	PENDING, COMPLETED, FAILED, REFUNDED
transactionId	Bank transaction reference number

Table 5: NOTIFICATIONS

Alert messages sent to users

Column	Meaning
id	Notification ID
user_id	Which user receives this notification
title	Notification title
message	Notification message
isRead	Has user read this notification
createdAt	When notification was created

Table 6: RATINGS

Reviews and star ratings after campaign completion

Column	Meaning
id	Rating ID
campaign_id	Which campaign this rating is for
rater_id	Who gave the rating (Brand or Influencer)
rated_id	Who is being rated (other person)
score	Star rating (1-5)
feedback	Written review

How Tables Connect

```
USERS (Admin, Brands, Influencers)
  ↓ (brand_id)
CAMPAIGNS (Created by Brands)
  ↓ (campaign_id)
SPONSORSHIP_REQUESTS (Influencers apply)
  ↓ (influencer_id, campaign_id)
PAYMENTS (Money transfer)
  ↓ (campaign_id, user_id)
RATINGS (Leave feedback)
  ↓ (from any user)
NOTIFICATIONS (Get alerts)
```

6 Authentication & Security

What is Authentication?

Authentication means **verifying who you are**. When you log in with email and password, you're telling the system "I'm John Smith". The system verifies you're actually John by checking the password. That's authentication.

Complete Login Flow

Step 1: User Enters Credentials

Frontend: User types email "john@gmail.com" and password "mypassword123"

Step 2: Frontend Sends to Backend

Frontend sends HTTP POST request: {"email":"john@gmail.com",
"password":"mypassword123"}

Step 3: Backend Receives Request

AuthController receives the request and calls AuthService.login()

Step 4: Database Lookup

AuthService asks database: "Do we have a user with email john@gmail.com?"

Step 5: Password Verification (BCrypt)

Backend takes user's password from database and compares it with typed password

Important: Passwords stored ENCRYPTED with BCrypt, so even database doesn't have original password!

Step 6: Create JWT Token if Valid

If password matches, backend creates a JWT token (like a session ID) containing:

- User ID
- User email
- User role (ADMIN, BRAND, INFLUENCER)
- Expiration time (24 hours)

Step 7: Send Token to Frontend

Backend sends JWT token back to frontend

Step 8: Frontend Stores Token

Frontend saves token in browser's localStorage (like browser notepad)

Step 9: Token Used in Future Requests

Every time frontend calls backend API, it includes the JWT token in headers.

Backend checks: "Is this a valid token? Has it expired? Does user have permission?"

What is JWT Token?

JWT (JSON Web Token) is like a concert ticket. When you buy a ticket, it has:

- Your name
- Concert date
- Seat number
- Signature (so nobody can fake it)

JWT works the same way. It has user information + digital signature. Only the server (who created it) can verify the signature is valid.

```
JWT Token Example: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5M
.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c Three parts separated
by dots (.) Part 1: Header (algorithm used) Part 2: Payload (user
info) - encoded but NOT encrypted Part 3: Signature - proves it
wasn't modified
```

Role-Based Access Control (RBAC)

Different users have different permissions. Admin can do everything, but Influencer can't delete users.

Action	Admin	Brand	Influencer
View all users	✓ Yes	X No	X No
Create campaign	✓ Yes	✓ Yes	X No
Apply to campaign	X No	X No	✓ Yes
Approve application	✓ Yes	✓ Yes	X No
Process payment	✓ Yes	✓ Yes	X No
Ban users	✓ Yes	X No	X No

Auth Interceptor (The Silent Guardian)

The Auth Interceptor is a piece of code that runs automatically before every HTTP request. It:

- 1. Checks if user is logged in (JWT token exists)
- 2. If yes, adds token to request header: Authorization: Bearer <token>
- 3. If request fails with 401 (unauthorized), logs user out
- 4. If response is error, catches it and shows error message

```
// Auth Interceptor Logic function intercept(request) { const token = localStorage.getItem('jwt_token'); if (token) { request.headers.set('Authorization', 'Bearer ' + token); } return send(request) .catch(error => { if (error.status === 401) { // Token expired or invalid logout(); navigate('/login'); } throw error; // Re-throw error for component to handle }); }
```

Security Best Practices Implemented

Security Measure	Implementation
------------------	----------------

Password Encryption	BCrypt with strength 10 - takes time to crack
Token Expiration	JWT expires after 24 hours - must log in again
Stateless Auth	No sessions stored on server - scales better
CORS Configuration	Frontend can only call backend from localhost:4200
Role Verification	@PreAuthorize checks user role before executing method
HTTPS Ready	SSL configuration available for production

7 Complete Data Flow

Example: Brand Creates Campaign

1. FRONTEND (Angular)
 - └ User fills campaign form
 - └ Clicks "Create Campaign" button
 - └ CampaignComponent.createCampaign() is called
2. SERVICE LAYER (Frontend)
 - └ CampaignService prepares data
 - └ Adds JWT token via Auth Interceptor
 - └ Makes HTTP POST request to: /api/brand/campaigns
3. NETWORK
 - └ Request travels to backend server
4. BACKEND - CONTROLLER
 - └ BrandController receives request at /api/brand/campaigns
 - └ Extracts data from request body
 - └ Gets current user from JWT token
 - └ Calls campaignService.createCampaign()
 - └ Waits for service response
5. BACKEND - SERVICE
 - └ campaignService.createCampaign() receives data
 - └ Validates data (name not empty?, budget > 0?)
 - └ Creates new Campaign object
 - └ Sets campaign.brand = currentUser
 - └ Calls campaignRepository.save(campaign)
 - └ Waits for database response
6. BACKEND - REPOSITORY
 - └ Converts Campaign object to SQL INSERT
 - └ Sends to MySQL database
7. DATABASE (MySQL)
 - └ Inserts new row in CAMPAIGNS table
 - └ Auto-generates campaign ID
 - └ Returns inserted campaign with ID
 - └ Sends back to repository
8. BACKEND - SERVICE
 - └ Receives saved campaign from database
 - └ Creates notification for brand
 - └ Calls notificationRepository.save()
 - └ Returns campaign object

9. BACKEND - CONTROLLER

- └─ Receives campaign from service
- └─ Converts to JSON format
- └─ Sends HTTP response with 200 OK status

10. NETWORK

- └─ Response travels to frontend

11. FRONTEND - SERVICE

- └─ Receives response
- └─ Updates local campaign list
- └─ Calls component's callback

12. FRONTEND - COMPONENT

- └─ Shows success message
- └─ Redirects to campaigns list
- └─ User sees their new campaign!

Example: Influencer Applies to Campaign

Step 1: Influencer clicks "Apply" button on campaign

Component calls: `sponsorshipService.applyToCampaign(campaignId, proposal)`

Step 2: Frontend sends HTTP POST to `/api/influencer/apply`

Body: `{ campaignId: 5, proposal: "I have 100K followers in fashion..." }`

Step 3: Backend `SponsorshipController` receives request

Extracts `campaignId` and `influencerId` from JWT token

Step 4: Service checks for duplicate applications

Query: "Does this influencer already have a pending/accepted application for this campaign?"

If YES → Reject with error "Already applied!"

If NO → Continue

Step 5: Create `SponsorshipRequest` object and save to database

Fields: influencer_id, campaign_id, status = PENDING, proposal, createdAt

Step 6: Create notification for brand

Message: "New application from [InfluencerName] for your [CampaignName] campaign"

Step 7: Return success response to frontend

Frontend shows: "Application submitted successfully!"

Example: Brand Approves Work & Triggers Payment

Step 1: Influencer submits work

POST /api/influencer/submitWork with links to posts/videos

Status updated: PENDING → WORK_SUBMITTED

Step 2: Brand reviews and approves

PUT /api/brand/approveWork/{sponsorshipId}

Sets: workCompletedAt = current timestamp

Step 3: Service triggers payment creation

Because workCompletedAt is now set, system automatically:

- Creates Payment object
- Amount = campaign.budget
- Status = PENDING (waiting for processing)
- Saves to database

Step 4: System sends notifications

- To Brand: "Work approved! Payment processing..."
- To Influencer: "Congratulations! Payment sent to your account"

Step 5: Update sponsorship status

SponsorshipRequest.status = COMPLETED

Campaign.status = COMPLETED (if all influencers done)

8 Components Explained in Simple Terms

What is a Component Really?

Imagine you're building a house. A component is like a room. Each room has:

- Walls and paint (HTML - visual structure)
- Light switches and appliances (TypeScript - functionality)
- Furniture and decoration (CSS - styling)

Your whole house (application) is made of rooms (components). Each component is independent but can communicate with others.

LoginComponent

What it looks like: Login form with email & password fields

What happens:

1. User enters email and password
2. Clicks "LOGIN" button
3. Component calls `authService.login(email, password)`
4. Service sends to backend
5. If successful, JWT token saved and user redirected to dashboard
6. If failed, error message shown

Key variables: email, password, isLoading, errorMessage

SignupComponent

What it looks like: Registration form

What happens:

1. User fills: name, email, password, role selection

2. Validation checks:

- Is email format correct?
- Is password strong enough?
- Are passwords matching?

3. If valid, calls authService.signup()

4. Backend creates user and returns JWT

5. Redirects to dashboard

Important: Password is NEVER saved as plain text. Backend encrypts it!

CampaignListComponent

What it looks like: Grid/list of all campaigns

What happens:

1. Component loads
2. Calls campaignService.getAllCampaigns()
3. Service sends GET request to /api/campaigns
4. Backend returns array of campaigns
5. Component displays in list/grid format
6. User can filter by status, platform, budget
7. Clicking a campaign shows details

Features: Search, sort, pagination

CampaignDetailComponent

What it looks like: Single campaign with full details

What happens:

1. Gets campaign ID from URL
2. Loads campaign details from service
3. Displays: name, description, budget, timeline, applications count
4. If user is influencer: Shows "Apply" button
5. If user is brand who created it: Shows "Edit" and "Delete" buttons

6. Shows list of applications received

Logic: Different buttons shown based on user role

CampaignFormComponent

What it looks like: Form to create/edit campaign

Form fields:

- Campaign name
- Description
- Platform (dropdown: Instagram, YouTube, TikTok, etc)
- Budget
- Start date
- End date
- Eligibility requirements

Validation:

- Name is required
- Budget must be positive number
- End date must be after start date
- Description minimum 50 characters

DashboardComponent

What it's for: Main page after user logs in

Admin Dashboard shows:

- Total users count
- Total campaigns
- Total revenue
- Recent activities
- List of all users

Brand Dashboard shows:

- My campaigns
- Applications received
- Pending approvals
- Revenue from this month

Influencer Dashboard shows:

- Applied campaigns
- Active applications
- Completed campaigns
- Total earnings

 PaymentComponent

What it's for: Process payments from brand to influencer

Steps:

1. Shows payment summary (amount, recipient, campaign)
2. Shows payment status
3. If PENDING, shows "Confirm Payment" button
4. Clicks button → Payment processed
5. Updates status to COMPLETED
6. Sends notification to influencer

Rules:

- Payment only allowed after work is completed
- No duplicate payments
- Amount cannot be changed

 NotificationComponent

What it's for: Show alerts to user

Types of notifications:

- "You have a new application"
- "Your application was approved!"

- "Payment received"
- "Campaign expires in 3 days"

Features:

- Badge showing unread count
- Click to mark as read
- Delete old notifications
- Real-time updates (if implemented)

★ RatingComponent

What it's for: Rate after campaign completion

How it works:

1. Shows after campaign marked as COMPLETED
2. User selects stars (1-5)
3. Writes feedback text
4. Clicks "Submit Review"
5. Rating saved to database
6. Both parties see ratings on profile

Purpose: Build reputation on platform



User Roles & Detailed Permissions

Why Different Roles?

Not everyone should be able to do everything. If influencers could mark campaigns as completed, they'd approve their own work and get paid without doing anything. That's why we have roles with different permissions.

Role 1: ADMIN

Who: Platform administrators (Cognizant employees)

Can DO:

- ✓ Create/edit/delete any campaign
- ✓ View all users
- ✓ Suspend/ban users
- ✓ View all payments
- ✓ Generate reports
- ✓ Manage platform settings
- ✓ View transaction history

Cannot DO:

- X Apply to campaigns (not a brand or influencer)
- X Receive payments (unless also assigned as brand)

Default credentials: admin@gmail.com / admin@123

Role 2: BRAND

Who: Companies that want to promote products (e.g., Nike, Zara, etc)

Can DO:

- ✓ Create campaigns

- ✓ Edit their own campaigns
- ✓ Delete their own campaigns
- ✓ View applications to their campaigns
- ✓ Approve/reject applications
- ✓ Request influencer to submit work
- ✓ Review and approve submitted work
- ✓ Process payments to influencers
- ✓ Rate influencers after campaign
- ✓ View payment history
- ✓ Edit profile and bio

Cannot DO:

- X Apply to other campaigns
- X View other brands' campaigns details
- X View other users' contact info
- X Delete accounts (only admins)

Default credentials: brand@gmail.com / brand@123

Role 3: INFLUENCER

Who: People with large social media following

Can DO:

- ✓ View all campaigns
- ✓ Apply to campaigns with proposal
- ✓ View their applications status
- ✓ Receive work approval requests
- ✓ Submit work (links to posts/videos)
- ✓ View accepted campaigns
- ✓ Receive payment notifications
- ✓ Rate brands after campaign
- ✓ View earnings dashboard
- ✓ Edit profile and bio
- ✓ Manage profile image

Cannot DO:

- X Create campaigns
- X Approve/reject applications
- X Process payments
- X View other influencers' applications
- X View brand's payment info

Default credentials: influencer@gmail.com / influencer@123

10 Complete Real-World Workflows

Workflow 1: Brand Creating & Launching Campaign

```
START
↓
Brand logs in with email & password
↓
Dashboard → Click "Create Campaign"
↓
Fill form:
  - Name: "Summer Fashion 2024"
  - Description: "Promote our new summer collection..."
  - Platform: Instagram
  - Budget: $5,000
  - Start date: June 1, 2024
  - End date: August 31, 2024
  - Requirements: "Minimum 10K followers, fashion niche"
↓
Click "Create Campaign"
↓
BACKEND PROCESS:
  Frontend sends HTTP POST to /api/brand/campaigns
  ↓
  BrandController validates user is BRAND role
  ↓
  CampaignService.createCampaign():
    - Validate all fields
    - Create Campaign object
    - Set brand = current user
    - Save to database
    - Create notification for brand
  ↓
  Return campaign with ID
  ↓
FRONTEND RESULT:
  Show success message
  ↓
  Redirect to campaign details page
  ↓
  Brand sees their campaign is LIVE!
  ↓
  Notifications sent to all influencers (via email/in-app)
  ↓
END
```

Workflow 2: Influencer Applying to Campaign

```
START
↓
Influencer logs in
↓
Dashboard → Browse Campaigns
↓
Sees "Summer Fashion 2024" campaign
↓
Clicks to view details
↓
Reads campaign details & requirements
↓
If interested, clicks "Apply Now"
↓
Modal/Form appears with:
  - Proposal text area
  - Previous portfolio links (optional)
  - Why you're interested
↓
Influencer fills proposal:
  "I have 50K Instagram followers in fashion niche.
  I can create 5 posts and 3 reels about your summer collection.
  My engagement rate is 8.5%. Here's my previous work: [links]"
↓
Clicks "Submit Application"
↓
BACKEND PROCESS:
  Check for duplicate applications
  IF duplicate EXISTS:
    → Show error: "You already applied to this campaign"
    → STOP
  ↓
  IF no duplicate:
    - Create SponsorshipRequest object
    - Status = PENDING
    - influencer_id = current user
    - campaign_id = campaign ID
    - Save to database
    - Create notification for brand
  ↓
FRONTEND RESULT:
  Show success: "Application submitted!"
  ↓
  Add to "My Applications" list
  ↓
  Status shows: PENDING
  ↓
BRAND RECEIVES:
  Dashboard notification: "New application from [InfluencerName]"
```

↓
Brand clicks to review application & influencer profile
↓
END (waiting for brand decision)

Workflow 3: Brand Hiring Influencer & Paying

```
START
↓
Brand receives application from influencer
↓
Reviews influencer's profile:
- Followers count
- Portfolio
- Previous ratings
- Engagement metrics
↓
If interested:
  Clicks "Approve" button
  ↓
  BACKEND UPDATES:
    SponsorshipRequest.status = ACCEPTED
    ↓
    Create notification to influencer:
    "Your application to 'Summer Fashion' was approved!
    Please submit your work by [deadline]"
    ↓
  INFLUENCER RECEIVES:
    Dashboard notification: "Approved for Summer Fashion campaign!"
    ↓
    Can now submit work
    ↓
    Influencer creates content:
    - Posts on Instagram
    - Takes screenshots/videos
    - Collects proof of work
    ↓
    Navigates to campaign → Submit Work
    ↓
    Fills work submission form:
    - Links to posts
    - Analytics screenshots
    - Description of work done
    ↓
    Clicks "Submit Work"
    ↓
    BACKEND:
      SponsorshipRequest.workSubmittedAt = current time
      Status = WORK_SUBMITTED
```

↓
Create notification for brand:
"Influencer submitted work for review"
↓
BRAND RECEIVES:
Dashboard notification
↓
Clicks to review work
↓
Checks:
- Did they complete all requirements?
- Is quality acceptable?
- Did they reach engagement targets?
↓
If satisfied, clicks "Approve Work"
↓
BACKEND PROCESS:
SponsorshipRequest.workCompletedAt = current time
Status = COMPLETED
↓
TRIGGER AUTOMATIC PAYMENT:
PaymentService.createPayment():
- amount = campaign.budget (\$5,000)
- status = PENDING
- campaign_id = campaign ID
- influencer_id = influencer ID
- brand_id = brand ID
- Save to database
↓
Create notifications:
- To brand: "Payment processing..."
- To influencer: "Work approved! Payment incoming..."
↓
BRAND PROCESSES PAYMENT:
Navigates to Payments → Pending Payments
↓
Clicks "Confirm Payment" for \$5,000
↓
BACKEND:
Payment.status = COMPLETED
Payment.paidAt = current time
Payment.transactionId = [bank reference]
↓
Create notification to influencer:
"Payment of \$5,000 received in your account!"
↓
INFLUENCER RECEIVES:
Payment notification
↓
Checks their earnings dashboard
↓
Sees: \$5,000 received ✓
↓

Workflow 4: Rating & Feedback

After payment confirmed:

Both brand and influencer can rate each other

INFLUENCER'S PERSPECTIVE:

↓

Clicks "Rate this Client"

↓

Rating form appears:

- Stars (1-5) - Influencer gives 5 stars
- Feedback: "Great company to work with!
Clear requirements, quick payments.
Would love to work again."

↓

Clicks "Submit Review"

↓

BACKEND: Rating saved

↓

BRAND'S PERSPECTIVE:

↓

Clicks "Rate this Influencer"

↓

Rating form appears:

- Stars (1-5) - Brand gives 4 stars
- Feedback: "Great deliverables!
High quality content.
Very professional.
Minor: small delay in final delivery."

↓

Clicks "Submit Review"

↓

BACKEND: Rating saved

↓

RESULTS:

Both profiles now show:

- Average rating
- All reviews
- Trust score increases

↓

Future clients can see these ratings!

↓

Reputation system built ✓

Key Concepts Summary

Important Concepts You MUST Understand

Concept	Simple Explanation	Real Example
Entity	Java class that represents database table	User.java = users table in database
Repository	Talks to database, returns objects	<code>userRepository.findByEmail("john@gmail.com")</code>
Service	Contains business logic & decisions	<code>authService.login()</code> validates password
Controller	Receives HTTP requests, calls services	POST /api/campaigns → CampaignController
Component	Frontend piece user interacts with	LoginComponent = login page
Service (Frontend)	Handles API calls and data sharing	<code>campaignService.getCampaigns()</code>
Dependency Injection	Automatic providing of needed objects	<code>private authService: AuthService</code> (auto-provided)

JWT Token	Proof that user is logged in	Sent with every request: Bearer [token]
Observable	Data that arrives over time	HTTP response arrives, component updates
CORS	Security rule: which domains can access API	Only localhost:4200 can call backend



Technical Deep-Dive Explanations

How Java & Spring Boot Handle Requests

```
// User clicks Apply button (Frontend)
campaignService.applyToCampaign(campaignId, proposal) // This calls
(Frontend Service) @Injectable() export class CampaignService {
constructor(private http: HttpClient) { }
applyToCampaign(campaignId, proposal) { // Prepare data const
requestBody = { campaignId: campaignId, proposal: proposal }; //
Make HTTP POST request // Auth Interceptor automatically adds JWT
token! return this.http.post('/api/influencer/apply', requestBody);
} } // HTTP Request reaches Backend Spring Boot Server // URL: POST
/api/influencer/apply // Headers: Authorization: Bearer [JWT_TOKEN]
// Body: {"campaignId": 5, "proposal": "..."} // Spring finds the
matching Controller @RestController
@RequestMapping("/api/influencer") public class InfluencerController
{ @PostMapping("/apply") public ResponseEntity<?> applyCampaign(
@RequestBody ApplyRequest request, @AuthenticationPrincipal
UserDetails userDetails // JWT extracted! ) { // Get current user
from JWT String userEmail = userDetails.getUsername(); User
influencer = userService.findByEmail(userEmail); // Call service
SponsorshipRequest result = sponsorshipService.applyCampaign(
influencer.getId(), request.getCampaignId(), request.getProposal()
); // Return response return ResponseEntity.ok(result); } } //
Service contains actual logic @Service public class
SponsorshipService { public SponsorshipRequest applyCampaign( Long
influencerId, Long campaignId, String proposal ) { // Check if
already applied SponsorshipRequest existing =
repository.findByIdInfluencerIdAndCampaignId(influencerId,
campaignId); if (existing != null) { throw new
RuntimeException("Already applied to this campaign!"); } // Fetch
entities from database User influencer =
userRepository.findById(influencerId).orElseThrow(); Campaign
campaign = campaignRepository.findById(campaignId).orElseThrow(); //
Create new sponsorship request SponsorshipRequest request = new
SponsorshipRequest(); request.setInfluencer(influencer);
request.setCampaign(campaign); request.setProposal(proposal);
request.setStatus(RequestStatus.PENDING);
request.setCreatedAt(LocalDateTime.now()); // Save to database
SponsorshipRequest saved = repository.save(request); // Create
notification for brand Notification notification = new
Notification(); notification.setUser(campaign.getBrand());
```

```

notification.setTitle("New Application");
notification.setMessage("Influencer " + influencer.getName() + "
applied to your " + campaign.getName() + " campaign");
notificationRepository.save(notification); return saved; } } //
Repository talks to database (Hibernate handles SQL) public
interface SponsorshipRequestRepository extends
JpaRepository<SponsorshipRequest, Long> { // This method
automatically generates SQL! // SELECT * FROM sponsorship_requests
// WHERE influencer_id = ? AND campaign_id = ? SponsorshipRequest
findByInfluencerIdAndCampaignId( Long influencerId, Long campaignId
); } // Response sent back to Frontend // Status: 200 OK // Body: {
// "id": 15, // "influencer": {...}, // "campaign": {...}, //
"proposal": "I have 100K followers...", // "status": "PENDING", //
"createdAt": "2024-05-23T10:30:00" // } // Frontend receives and
shows success message!

```

How JWT Authentication Works (Technical)

```

// LOGIN PROCESS Frontend sends: POST /api/auth/login Headers:
Content-Type: application/json Body: { "email": "john@gmail.com",
"password": "mypassword123" } // Backend AuthController
@PostMapping("/login") public ResponseEntity<?> login(@RequestBody
LoginRequest request) { // Authenticate using Spring Security
Authentication auth = authenticationManager.authenticate( new
UsernamePasswordAuthenticationToken( request.getEmail(),
request.getPassword() ) ); // If password wrong, exception thrown
here // Authentication succeeds only if password matches UserDetails
userDetails = auth.getPrincipal(); // Generate JWT token String
token = jwtTokenProvider.generateToken(userDetails); return
ResponseEntity.ok( new LoginResponse(token, userDetails) ); } // JWT
Token Generation public String generateToken(UserDetails
userDetails) { Date now = new Date(); Date expiryDate = new
Date(now.getTime() + expiration); // 24 hours from now return
Jwts.builder() .setSubject(userDetails.getUsername()) // Subject =
email .claim("roles", userDetails.getAuthorities()) // Save roles
.setIssuedAt(now) // When created .setExpiration(expiryDate) // When
expires .signWith(SignatureAlgorithm.HS512, secretKey) // Sign with
secret .compact(); } // Token looks like: //
eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9. //
eyJzdWIiOiJqb2huQGdtYWlsLmNvbSIsInJvbGVzIjpbIkFETUluIl0sIm1hdCI6MTYwMz
// 2BfUlBb1Sm-U7VqJ8nWlPq4xQ9YzW3z5z5z... // USING TOKEN IN REQUESTS
Frontend stores token in localStorage Every HTTP request includes:
Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9... //
Auth Interceptor adds token automatically @Injectable() export class
AuthInterceptor { intercept(req: HttpRequest<any>, next:
HttpHandler) { const token = localStorage.getItem('jwt_token'); if

```

```

(token) { req = req.clone({ setHeaders: { 'Authorization': `Bearer
${token}` } }); } return next.handle(req).pipe( catchError(error =>
{ if (error.status === 401) { // Token expired
localStorage.removeItem('jwt_token'); router.navigate(['/login']); }
return throwError(error); }) ); } } // BACKEND VALIDATES TOKEN // On
every request, JwtAuthenticationFilter runs @Component public class
JwtAuthenticationFilter extends OncePerRequestFilter { @Override
protected void doFilterInternal(...) { String token =
getTokenFromRequest(request); if (token != null) { try { // Verify
token signature Claims claims = Jwts.parser()
.setSigningKey(secretKey) .parseClaimsJws(token) .getBody(); // If
parsing succeeds, token is valid! String username =
claims.getSubject(); // Get user and create authentication
UserDetails userDetails =
userService.loadUserByUsername(username); Authentication auth
= new UsernamePasswordAuthenticationToken( userDetails, null,
userDetails.getAuthorities() ); // Set in Spring Security context
SecurityContextHolder.getContext().setAuthentication(auth); } catch
(ExpiredJwtException e) { // Token expired logger.error("Token
expired"); } catch (JwtException e) { // Token tampered/invalid
logger.error("Invalid token"); } } } } // If @PreAuthorize check
fails @GetMapping("/admin/users") @PreAuthorize("hasRole('ADMIN')")
public List<User> getAllUsers() { // Only ADMIN role can access //
Others get 403 FORBIDDEN response }

```

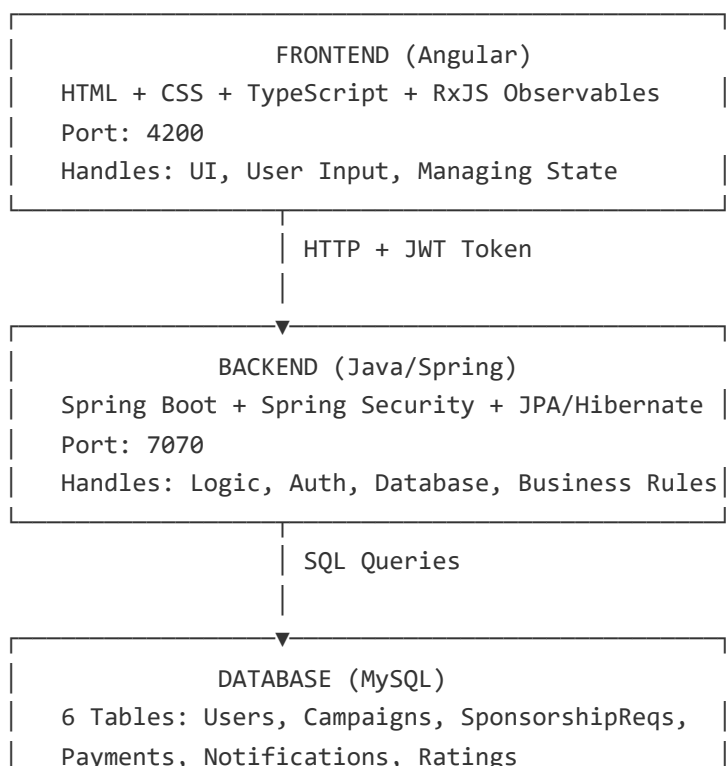


Final Summary & Quick Reference

Congratulations! You Now Understand:

- ✓ What every technology does
- ✓ How backend processes requests
- ✓ How frontend displays data
- ✓ How they communicate via HTTP & JWT
- ✓ How database stores information
- ✓ How authentication protects data
- ✓ How different components interact
- ✓ What each service is responsible for
- ✓ How the complete workflow works end-to-end

Technology Stack at a Glance



Port: 3306

Handles: Data Storage, Queries, Relationships

Important Points to Remember

Point	Remember
Passwords	ALWAYS encrypted with BCrypt. Never stored as plain text
JWT Tokens	Contains user info, expires after 24 hours. Lost if browser cleared
Role-Based Access	Enforced both Frontend (Guards) and Backend (@PreAuthorize)
Database Relationships	Foreign keys connect tables. Prevent orphaned data
Payment Rules	Payment only after work marked COMPLETED. No duplicates
Duplicate Prevention	Influencer can't apply twice to same campaign
Notifications	Auto-triggered on status changes. Keep users updated
CORS	Security feature. Only localhost:4200 can access backend

Quick Interview Answers

Q: What does the application do?

A: It's a platform connecting brands needing social media promotion with influencers who can help. Brands create campaigns, influencers apply, they complete work together, and the platform handles payments and ratings.

Q: How does authentication work?

A: Users login with email & password. Backend verifies password (encrypted with BCrypt), creates JWT token, and sends it to frontend. Every request includes this token, which backend validates to confirm user identity.

Q: How do frontend and backend communicate?

A: Via HTTP requests/responses. Frontend sends JSON data, backend processes and returns JSON. Auth Interceptor automatically adds JWT token to every request.

Q: What prevents influencer from applying twice?

A: Service checks database before creating application. If record exists with same influencer_id and campaign_id, it rejects with error.

Q: When is payment triggered?

A: When brand clicks "Approve Work", it sets workCompletedAt timestamp. The service automatically creates Payment record with PENDING status.

Q: What is the purpose of roles?

A: They restrict what each user type can do. Admin manages platform, Brand creates campaigns & pays, Influencer applies & completes work. Prevents misuse.

Q: How is data protected?

A: Passwords encrypted (BCrypt), JWT tokens validate on every request, role checks prevent unauthorized access, CORS restricts domain access, database has foreign key constraints.

Sponsorship App - Complete Learning Guide

Created for Cognizant Interim Evaluation

Generated: May 23, 2024

This comprehensive guide covers everything from basic concepts to technical deep-dives. Use it to prepare for interviews and demonstrate your expertise!